COMPUTER GRAPHICS PROJECT REPORT
SPRING 2026

Student ID:	220303044
Full Name:	İsmail TATLI

Project Github Link:	https://github.com/ismailtatli/target-hunter
GitHub Pages Link:	https://ismailtatli.github.io/target-hunter/

INTRODUCTION

What is your game?

Target Hunter is a browser-based 2D arcade shooting game developed with HTML5 Canvas, CSS, and JavaScript. In the game, the player controls a crosshair and shoots enemy targets while avoiding civilian targets. The game includes different target types, a scoring system, combo mechanics, level progression, time bonuses, freeze bonuses, and a game over screen with final statistics.

The project was designed as a complete and playable computer graphics game. It includes a clear start screen, continuous gameplay, increasing difficulty, real-time animation, interactive mouse input, and visual effects such as particles, glow effects, target animations, and motion-based feedback.

This is not just a visual demo; it is a complete playable game with a start-play-end loop, continuous interaction, scoring, combo, levels, increasing difficulty, and visible computer graphics concepts.

What is the objective of the game?

The objective of the game is to score as many points as possible before the time runs out or the player loses all lives. The player must shoot green enemy targets and orange elite targets to gain points, avoid red civilian targets, and collect blue and ice bonus targets when they appear.

As the player earns points, the level increases. Each new level makes the game more challenging by increasing target speed, changing target sizes, and increasing target pressure on the screen. The player must react quickly, aim accurately, and manage the combo system to achieve a higher score.

GAME DESCRIPTION

How does the game work?

The game starts with a main menu where the player enters a callsign and presses the Start Game button. After the game begins, targets appear on the canvas and move around the screen. The player uses the mouse to aim and click on targets.

Green enemy targets give normal points and increase the combo multiplier. Orange elite targets give higher points. Blue chrono targets add extra time and points. Ice crystal targets freeze all targets for a short time. Red civilian targets must be avoided because they reduce the player's score, remove one life, and reset the combo.

The game includes five different target types. Green targets are the main enemy targets and give normal score. Red civilian targets must be avoided because they reduce score, remove one life, and reset the combo. Orange elite targets are harder to hit but give higher score. Blue chrono targets give extra time and additional points. Ice crystal targets freeze all targets for 3 seconds and help the player control crowded situations.

The game uses a level system. Every 300 points, the player reaches a new level. Higher levels increase the difficulty by making the game faster and more intense. The game ends when the timer reaches zero or the player loses all lives. At the end, the game shows the final score, level, best combo, and rank.

What are the rules?

The main rules of the game are:

1. Shoot green enemy targets to gain points.
2. Shoot orange elite targets to gain more points.
3. Avoid red civilian targets because they decrease score, remove one life, and reset the combo.
4. Shoot blue chrono targets to gain extra time and points.
5. Shoot ice crystal targets to freeze all targets for 3 seconds.
6. Build combo by hitting targets in a row.
7. The combo can increase from ×1 up to ×8.
8. The player must hit another valid target within 2 seconds to keep the combo active.
9. Missing a shot, hitting a civilian, letting an enemy escape, or waiting too long resets the combo.
10. Every 300 points increases the level.
11. Higher levels increase speed, pressure, and difficulty.
12. The game ends when time reaches zero or the player loses all lives.
13. The Space key pauses and resumes the game.

GRAPHICS CONCEPTS USED

This project uses several computer graphics concepts covered in the course.

1. HTML5 Canvas Rendering:
The main game area is drawn using the HTML5 Canvas API. All targets, effects, particles, crosshair elements, background visuals, and gameplay animations are rendered dynamically on the canvas.

2. Animation Loop:
The game uses a real-time animation loop with requestAnimationFrame. This loop continuously updates target positions, redraws the canvas, checks collisions, updates timers, and renders visual effects. This creates smooth and continuous gameplay.

3. 2D Transformations:
The project uses 2D transformations such as translate, rotate, and scale. These transformations are used when drawing animated targets, rotating visual elements, radar effects, and crosshair feedback. Each target is drawn relative to its own position, which makes the drawing logic cleaner and more flexible.

4. Collision Detection:
Collision detection is used to determine whether the player clicked on a target. The game calculates the distance between the mouse click position and the target center. If the

distance is smaller than the target radius, the hit is accepted. This is essential for the shooting mechanic.

5. Particle Effects:
The game includes particle-based explosion effects when targets are hit or missed. Small particles are created with different positions, velocities, colors, and lifetimes. Their opacity decreases over time, creating a visual explosion effect.

6. Gradients, Shadows, and Glow Effects:
The visual style uses radial gradients, glow effects, shadows, and transparent colors. These effects make the targets, explosions, HUD, and menu screen look more polished and visually clear.

7. Motion and Difficulty Scaling:
Targets move with different speeds and directions. As the level increases, the movement speed and target pressure increase. This creates meaningful progression and prevents the game from feeling like a short demo.

8. User Interaction:
The player interacts with the game using mouse movement, mouse clicks, text input, and the Space key. Mouse movement controls the crosshair, clicking shoots targets, the input field sets the player name, and Space pauses or resumes the game.

9. UI and Game State Management:
The game includes multiple states such as start screen, active gameplay, pause screen, freeze state, and game over screen. These states are visually represented using HTML/CSS overlays and controlled through JavaScript.

10. Visual Feedback:
The game gives immediate visual feedback through score popups, combo flashes, target explosions, freeze effects, level-up banners, and HUD updates. This makes the gameplay more understandable and engaging.

The game is designed as a complete gameplay system instead of a simple visual demonstration. It includes a start screen, active gameplay, pause state, game over state, continuous player interaction, increasing difficulty, scoring, combo progression, and multiple animated target types.

SCREENSHOTS

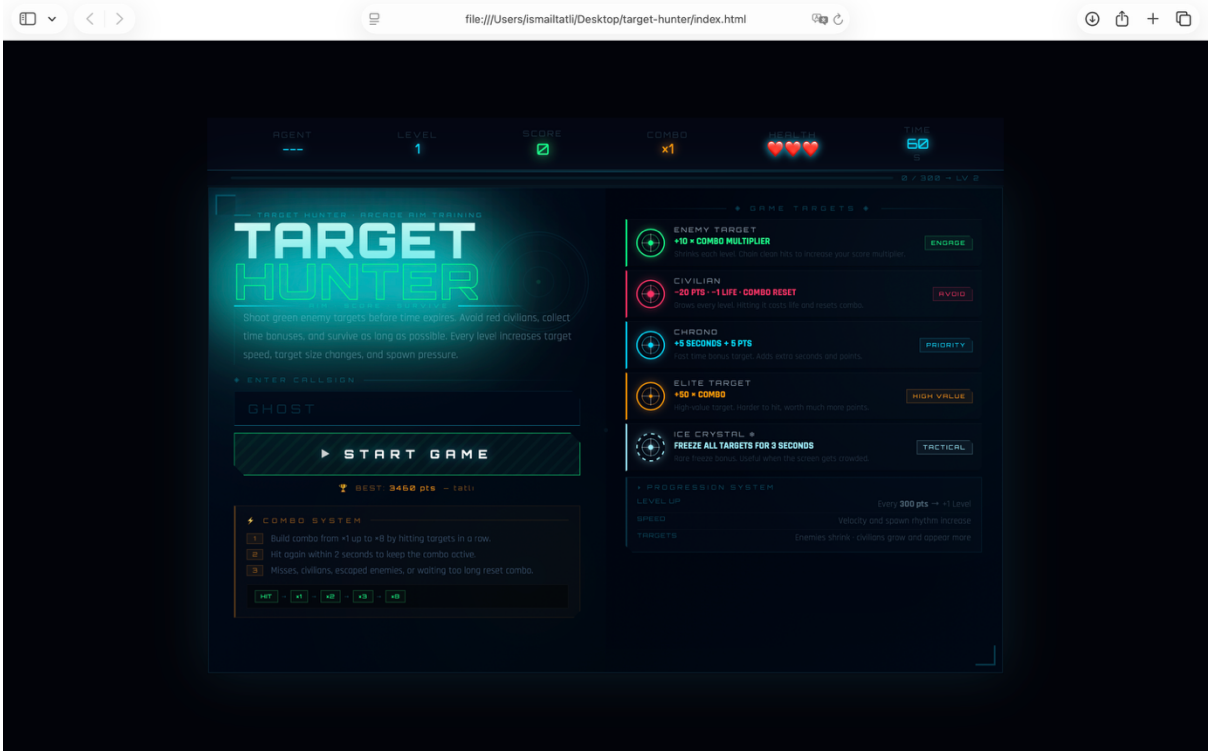

Figure 1. Start screen of Target Hunter with callsign input, combo explanation, target guide, and Start Game button.

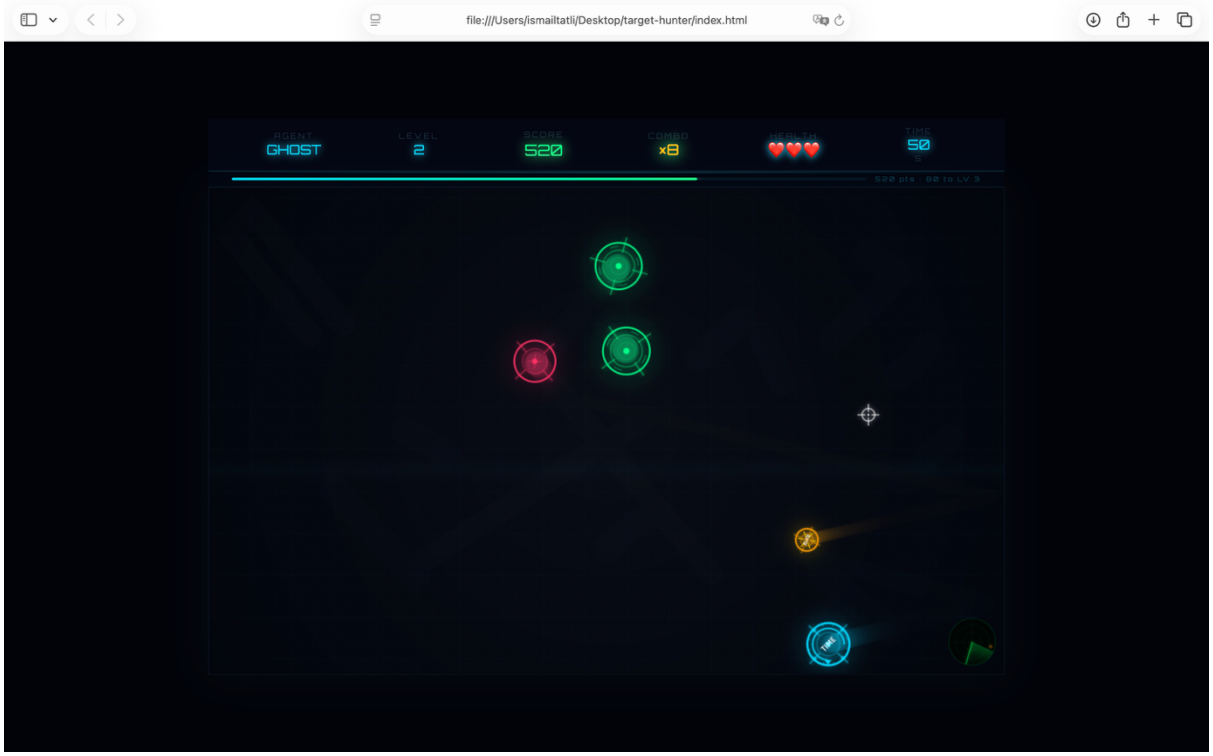

Figure 2. Main gameplay screen showing moving targets, HUD, timer, score, lives, level, and combo system.

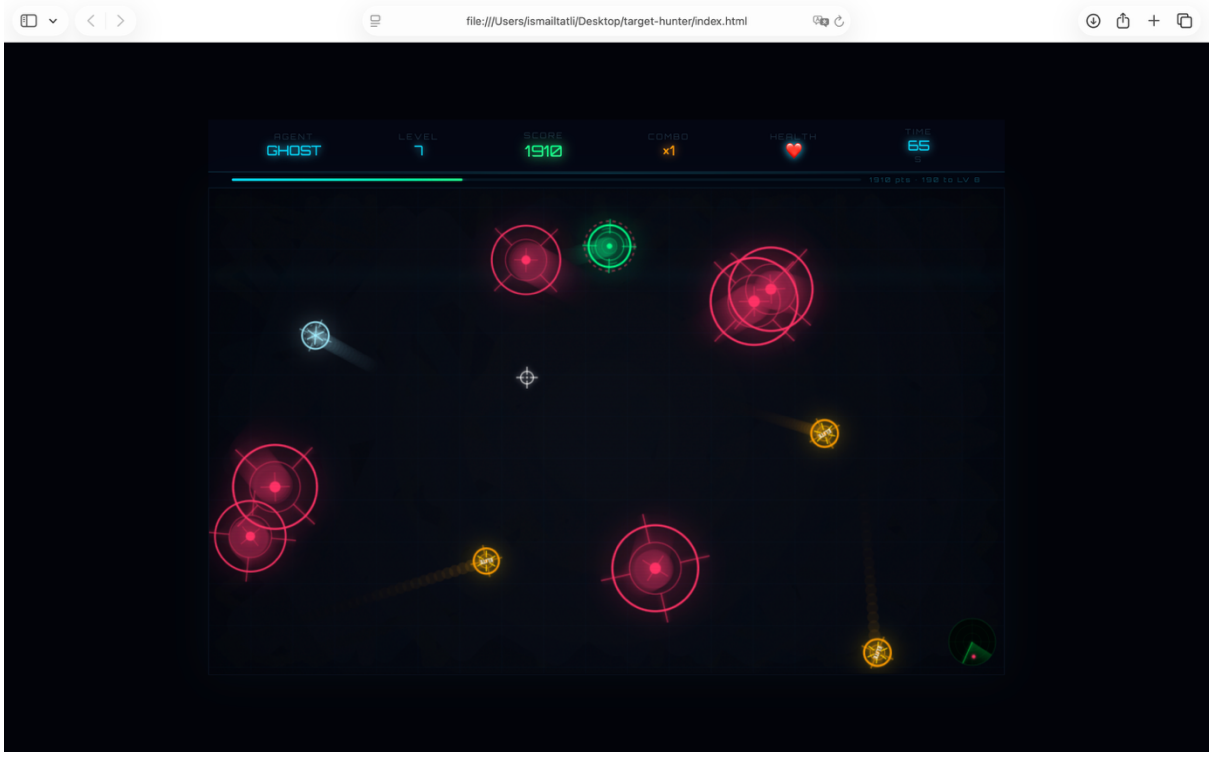

Figure 3. Gameplay during higher levels where target speed, target pressure, and difficulty increase.

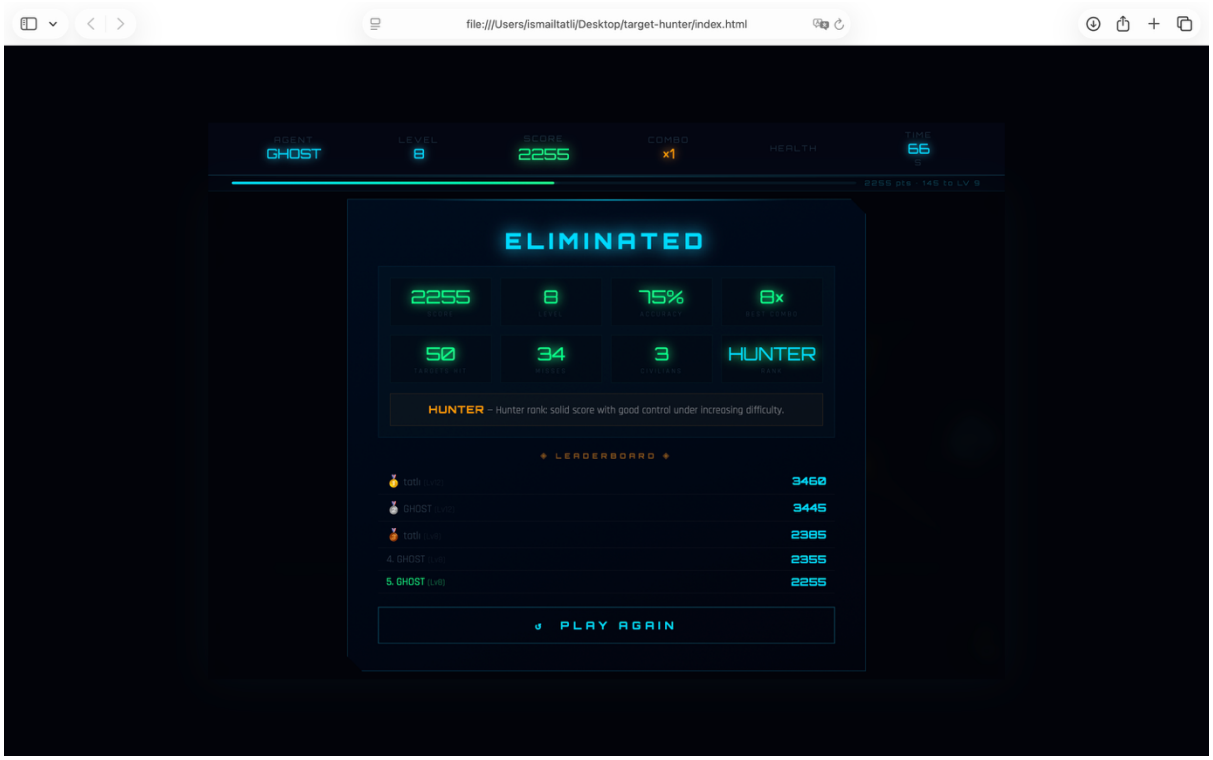

Figure 4. Game over screen showing final score, level, best combo, rank, and leaderboard information.

CONTRIBUTIONS

Individual Contribution

Ismail Tatlı designed and implemented the full Target Hunter game. His contributions include game idea development, HTML structure, CSS-based user interface design, JavaScript game logic, canvas rendering, target movement, collision detection, scoring system, combo system, level progression, pause system, visual effects, and GitHub Pages deployment.

CONCLUSION

What did you learn?

During this project, I learned how to build a complete interactive game using HTML5 Canvas and JavaScript. I improved my understanding of real-time rendering, animation loops, collision detection, mouse interaction, particle effects, and visual feedback systems.

I also learned how gameplay design and computer graphics concepts work together. For example, transformations and animations make the targets visually dynamic, while collision detection and input handling make the game interactive. In addition, I learned how to organize a browser-based project and deploy it using GitHub Pages.

What would you improve if you had more time?

If I had more time, I would improve the game by adding more target types, sound settings, difficulty modes, mobile support, and more detailed animations. I would also add a better leaderboard system, more background variations, and additional levels with different visual themes.

Another possible improvement would be adding a tutorial mode so that new players can learn the target types and combo system before starting the actual game.